Sử dụng model pretrained

Nạp đúng model và chạy dự đoán đầu tiên

Kho ảnh câu lạc bộ

Ít ảnh có nhãn, muốn xếp nhóm để tìm lại.

Có model đã được công bố. Ta cần gì ngoài tên model?

Cần thêm: cách xử lý ảnh đầu vào, bảng nhãn, đúng phiên bản.

Sai một thứ thì code vẫn chạy, nhưng kết quả sai.

Mục tiêu

- Phân biệt kiến trúc, trọng số, bộ xử lý đầu vào và tập nhãn của một model.
- Nạp model từ máy, chạy dự đoán theo batch và kiểm kết quả.
- Ước lượng bộ nhớ cho trọng số.
- Chuẩn bị file model để chạy khi không có Internet.

01

Một gói model gồm những gì

Kiến trúc khác checkpoint

`nn.Linear(2, 3)` tạo các phép toán và trọng số ban đầu.

Pretrained cần **trọng số đã học** từ nhiệm vụ trước.

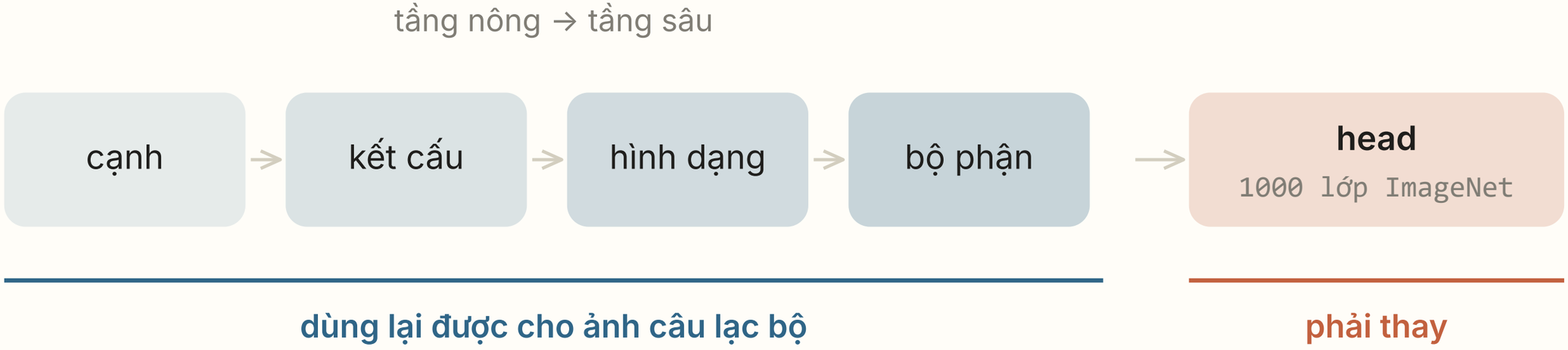
Processor quyết định input:

- Ảnh: resize, RGB, chuẩn hóa.
- Chữ: tokenizer, special token, padding.
- Head: nhãn và thứ tự lớp.

Vì sao trọng số cũ có ích?

- Phân biệt hơn một triệu ảnh của 1000 lớp buộc mạng học cạnh, kết cấu, hình dạng.
- Những đặc trưng ấy có trong mọi ảnh, kể cả ảnh câu lạc bộ.
- Chỉ **head** gắn chặt với 1000 lớp cũ.

Phần dùng lại, phần phải thay



Các tầng đầu học đặc trưng chung dùng lại được; chỉ head cuối gắn với 1000 lớp cũ.

HỎI NHANH

Có kiến trúc ResNet, và trọng số được khởi tạo ngẫu nhiên. Mô hình này đã là một model pretrained chưa?

A Rồi, vì tên kiến trúc trùng với model đã công bố

B Rồi, sau khi gọi `model.eval()` để suy luận

C Chưa, vì chưa nạp trọng số đã học

D Chưa, vì phải fine-tune trên dữ liệu mới trước

Shape trước khi chạy

Loại input	Shape
Ảnh	`(B, C, H, W)`
Token IDs	`(B, L)`
Logits phân loại	`(B, K)`

: batch · : lớp · : độ dài token.

Ảnh có kênh, chiều cao và chiều rộng .

ImageNet: chia 255, rồi chuẩn hóa mean/std từng kênh.

HỎI NHANH

Processor của checkpoint nhận pixel từ 0 đến 255, tự chia cho 255, rồi trừ mean và chia std. Code của đội lại đã chia cho 255 trước khi gọi processor. Chuyện gì xảy ra?

A Báo lỗi kiểu dữ liệu, vì input không còn là số nguyên

B Chạy bình thường, nhưng ảnh nào cũng gần như một màu

C Kết quả y hệt, vì chia hai lần vẫn giữ thứ tự các pixel

D Chỉ chậm hơn một chút, còn dự đoán không đổi

02

Kiểm bằng gói nhỏ

Một gói nhỏ, đủ các bước

Trọng số đặt tay để kiểm được từng con số.

```
model.load_state_dict(
    loaded["state_dict"], strict=True)
model.eval()
print("shape W:", tuple(model.weight.shape))
```

KẾT QUẢ

```
shape W: (3, 2)
```

- `strict=True`: kiểm tên và shape.
- `map_location="cpu"`: mở trên CPU.
- `weights_only=True`: chỉ mở tensor, số, chuỗi.
- Gói còn có mean, scale và nhãn.

HỎI NHANH

Checkpoint được lưu từ một model bọc trong `DataParallel`, nên các khóa là `module.weight` và `module.bias`, trong khi model mới chờ `weight` và `bias`. Đội gọi `load_state_dict(sd, strict=False)`. Kết quả là gì?

A PyTorch tự bỏ tiền tố `module.` rồi nạp đủ

B Báo lỗi ngay khi nạp, vì tên khóa không khớp

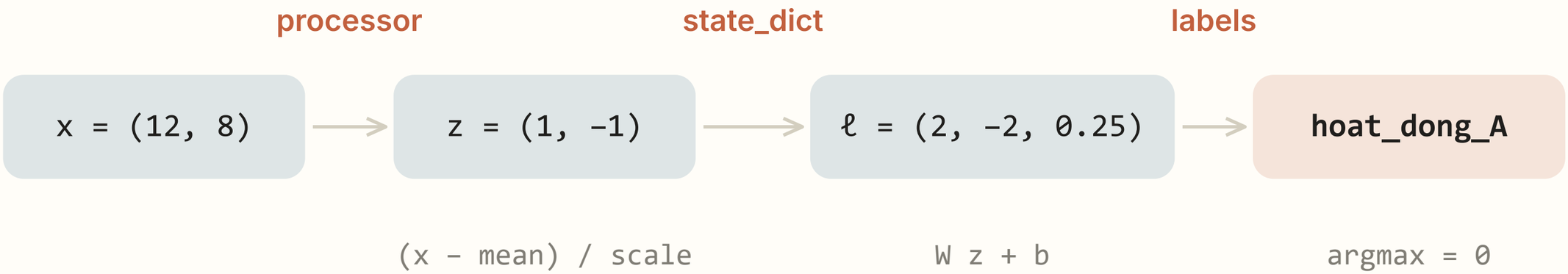
C Nạp được `bias`, còn `weight` vẫn ngẫu nhiên

D Không lỗi gì, nhưng model vẫn giữ trọng số khởi tạo

Một mẫu qua gói nhỏ

Đây là logits của một mẫu. Lớp 0 thắng; argmax không cần softmax.

Ba phần của gói, ba bước



thiếu hay lệch một phần nào, chương trình vẫn chạy mà trả lời sai

Một mẫu qua ba phần của gói: processor, trọng số, bảng nhãn.

Đọc logits

```
with torch.inference_mode():
    logits = model(process(raw))
    predicted = logits.argmax(dim=1)
print(logits.numpy().round(2))
```

KẾT QUẢ

```
[[ 2.   -2.    0.25]
 [-2.    2.    0.25]
 [ 0.    0.    0.25]
 [ 4.   -4.    0.25]
 [-1.    1.    0.25]]
```

Hàng thứ ba chỉ còn bias: input ở đúng trung bình.

HỎI NHANH

`logits` có shape `(5, 3)`. Muốn mỗi mẫu có một nhãn, ta dùng gì?

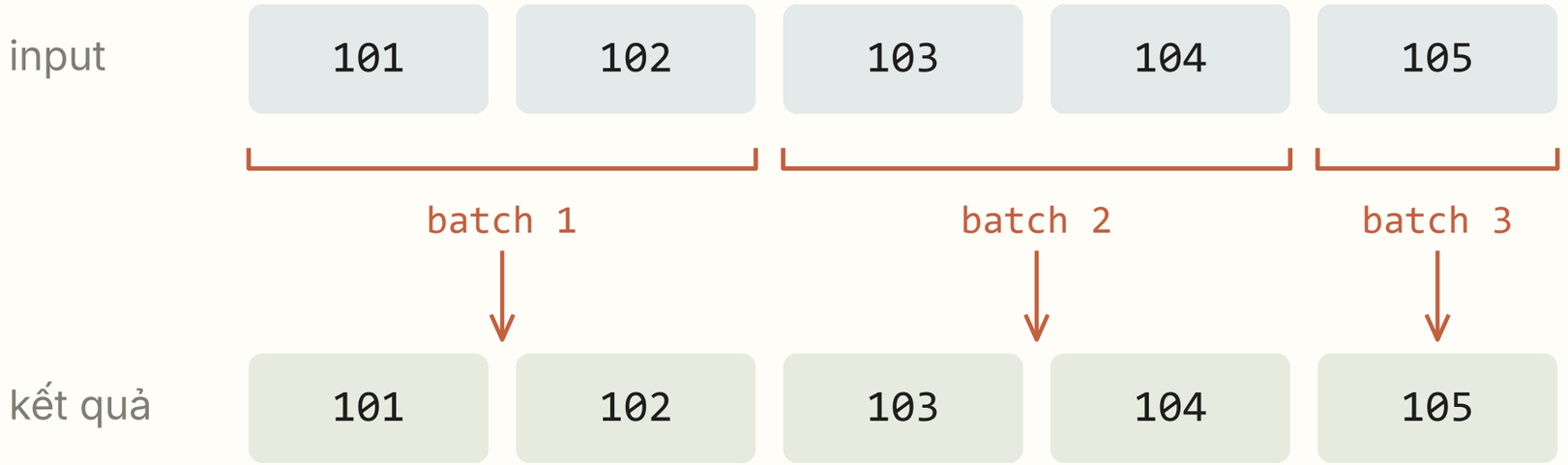
A `logits.argmax(dim=1)`

B `logits.argmax(dim=0)`

C `logits.argmax()`

D `logits.max(dim=1).values`

Batch cuối không được mất



batch 2, 2, 1 nối lại đúng thứ tự; giữ id theo từng hàng

Năm mẫu chia batch 2, 2, 1; kết quả nối lại theo đúng thứ tự id.

Hai phép kiểm khác nhau

```
print("batch khop:",
      torch.allclose(batched, logits))
print("NumPy khop:",
      np.allclose(batched.numpy(), expected))
print("so mau:", len(batched))
```

KẾT QUẢ

```
batch khop: True
NumPy khop: True
so mau: 5
```

Phép so batch kiểm việc ghép hàng.

Phép so NumPy kiểm công thức bằng một đường tính độc lập.

ID phải đi cùng mỗi output.

`eval()` và `inference_mode()` kiểm hai việc khác nhau

`model.eval()`:

Dropout và BatchNorm chuyển cách hoạt động.

`torch.inference_mode()`:

Không theo dõi gradient cho suy luận.

Nó không tự gọi `eval()`.

BatchNorm cần `eval()`

- Train mode: chuẩn hóa bằng thống kê **của batch**, đầu ra của một ảnh phụ thuộc ảnh đi cùng.
- Eval mode: dùng running statistics trong checkpoint, chia batch kiểu gì cũng như nhau.

HỎI NHANH

CNN pretrained có BatchNorm. Đội quên gọi `eval()`, nhưng có `inference_mode()`. Chạy cùng tập test với batch 32 rồi với batch 8, hai file nộp khác nhau ở vài dòng. Vì sao?

A `inference_mode()` làm phép tính không còn tất định

B Ở chế độ train, BatchNorm dùng thống kê của từng batch

C Batch cuối thiếu mẫu nên các dòng bị lệch hàng

D Batch nhỏ làm trọng số được cập nhật nhiều lần hơn

HỎI NHANH

Model đã gọi `eval()`, nhưng vòng dự đoán không bọc trong `inference_mode()` hay `no_grad()`. Tham số vẫn có `requires_grad=True`, còn input thì không. Câu nào đúng?

A Không ghi graph, vì `eval()` đã tắt autograd

B Không ghi graph, vì input không cần gradient

C Vẫn ghi graph, nhưng chỉ khi chạy trên GPU

D Vẫn ghi graph, vì `eval()` không đụng tới autograd

03

Bộ nhớ và model thật

Tính phần bộ nhớ trọng số

là số tham số; là byte mỗi tham số: FP32 dùng 4, FP16 dùng 2.

Một MiB bằng byte.

Model 100 triệu tham số:

Precision	Trọng số thô
FP32	381.47 MiB
FP16	190.73 MiB

Chưa tính input, activation, buffer, optimizer.

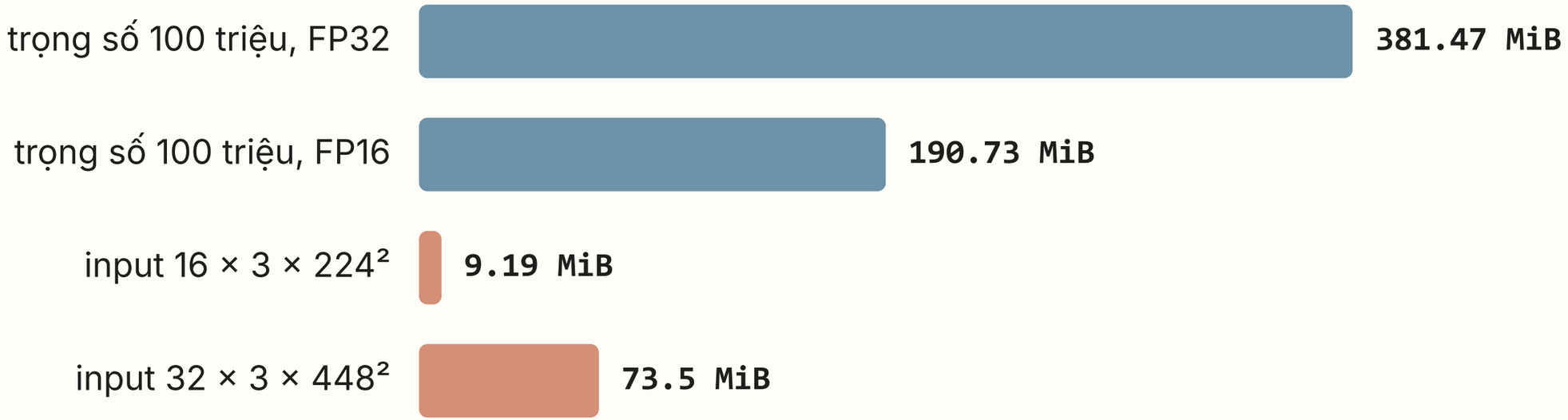
Input và activation theo

Batch 16 ảnh RGB , `float32`:

Gấp đôi cạnh ảnh tốn ngang gấp bốn batch.

Đo một batch nhỏ trên đúng thiết bị rồi tăng dần.

Trọng số và input



chưa tính activation, gradient, optimizer và bộ nhớ dự trữ

Gấp đôi batch và gấp đôi cạnh ảnh thì input gấp 8 lần.

HỎI NHANH

Một batch 16 ảnh RGB kiểu `float32` chiếm 9.19 MiB. Đổi sang batch 32 và ảnh thì tensor input chiếm bao nhiêu?

A 18.38 MiB

B 36.75 MiB

C 73.50 MiB

D 147.00 MiB

Đọc model card trước

- Học trên dữ liệu gì, input nào, nhãn nào, license nào.
- Head ImageNet chỉ chọn trong 1000 lớp cũ.
- Dùng lại các tầng dưới: 4.2 lấy đặc trưng, 4.3 học head mới.

HỎI NHANH

Đội đưa ảnh câu lạc bộ vào một ResNet học trên ImageNet, còn nguyên head, rồi lấy argmax trên 1000 logits. Kết quả là gì?

A Nhãn của câu lạc bộ, nếu ảnh đủ rõ và đúng cỡ

B Một lớp của ImageNet, không phải nhãn câu lạc bộ

C Báo lỗi shape, vì 1000 lớp không khớp ba nhãn mới

D Một nhãn ngẫu nhiên, vì model chưa thấy ảnh này

Dùng model thật offline

Lượt có mạng:

- Đọc model card và license.
- Chốt revision commit.
- Ghi hash các tệp.

Lượt offline:

- Kiểm hash, rồi nạp model và processor từ máy (`local_files_only=True`).
- Chạy dữ liệu nhỏ, kiểm shape và ID.

Rồi mới chấm validation.

HỎI NHANH

Hai tuần sau, đội tải lại cùng model ID từ nhánh `main`, chạy năm ảnh kiểm ở chế độ eval, và thấy logits khác lần trước ngay từ chữ số đầu. Nên kiểm gì đầu tiên?

A Commit và SHA-256, so với `ASSET.json` cũ

B Seed của NumPy và PyTorch trong notebook

C Batch size, vì đổi batch là đổi logits

D Phiên bản CUDA, vì GPU khác cho số khác

Hai lỗi processor trên ảnh thật

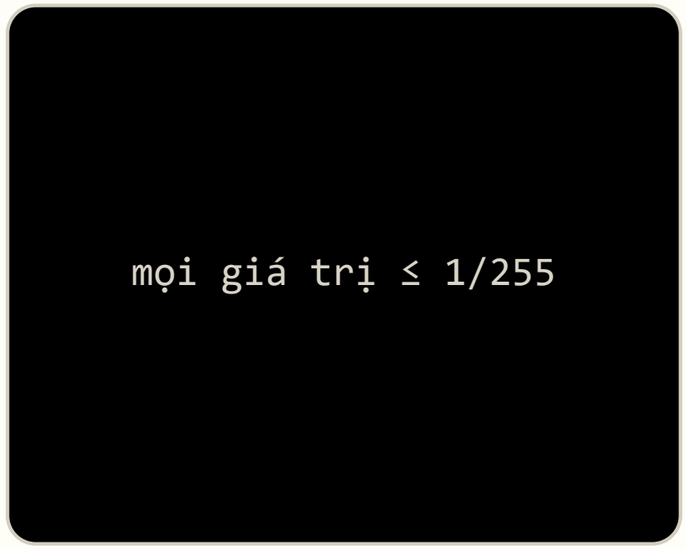
RGB đúng



đọc bằng cv2 (BGR)



chia 255 hai lần



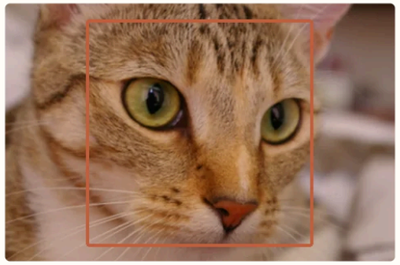
shape, dtype đều đúng: không có lỗi nào báo ra

Đọc bằng cv2 thì đỏ và lam bị đảo; chia 255 hai lần thì ảnh gần như một màu.

Ảnh thật qua processor và model



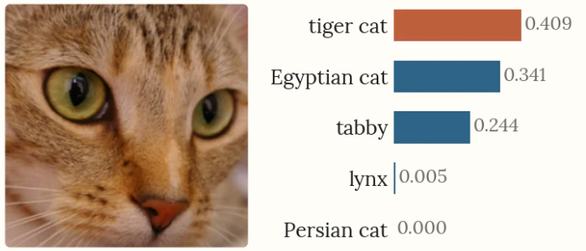
ảnh gốc 300 × 451 × 3, giá trị 0–255



cắt giữa một ô 224 × 224



chia 255, trừ mean, chia std từng kênh R, G, B: số nằm quanh 0



ResNet-18 trả 1000 xác suất; 5 lớp cao nhất cộng lại 1.000

Ảnh mèo qua đúng tiền xử lý của ResNet-18: ba lớp mèo đứng đầu.

Khi đi thi

- Đọc quy định về model, Internet, thư viện.
- Đầu giờ: nạp offline, chạy một batch, nộp thử.
- Processor giống hệt ở validation và test.

04

Luyện đề

LUYỆN ĐỀ 1

ResNet-50 nhận input `(B, 3, 224, 224)`. Phần đầu, gồm conv stride 2 rồi max-pool stride 2, đưa ảnh về . `layer1` giữ và cho ra 256 kênh. Khối đầu của `layer2` có conv stride 2 trên 128 kênh, rồi conv mở ra 512 kênh. Đầu ra của `layer2` có shape gì?

A `(B, 128, 28, 28)`

B `(B, 512, 28, 28)`

C `(B, 512, 56, 56)`

D `(B, 28, 28, 512)`

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 8.2

LUYỆN ĐỀ 2

Biểu thức nào đếm đúng số tham số học được của ResNet-50?

A
```
len(list(model.parameters()))
```

B
```
sum(t.numel() for t in
model.state_dict().values())
```

C
```
sum(p.numel() for p in
model.parameters())
```

D
```
sum(p.numel() * p.element_size()
for p in model.parameters())
```

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 8.1

LUYỆN ĐỀ 3

Trên Colab, đội lưu `torch.save(model, "m.pt")`, tức lưu cả đối tượng model. Máy chấm dùng PyTorch mới, và `torch.load("m.pt")` báo lỗi “Weights only load failed”. Lần sau nên lưu thế nào cho bền?

A Lưu `state_dict()` kèm config, rồi dựng model và nạp vào

B Gọi `model.save("m.pt")` như trong Keras

C Vẫn dùng `torch.save(model)`, và nạp với `weights_only=False`

D Chỉ lưu dự đoán trên validation thay cho model

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 134

Chốt ý

Một model dùng được gồm kiến trúc, trọng số, processor và bảng nhãn; thiếu một thứ là kết quả sai mà không báo lỗi.

Chạy theo batch phải giữ ID; đo bộ nhớ trên thiết bị thật; model tải về phải nạp lại được khi không có mạng.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: đổi batch size, kiểm thứ tự output không đổi.